List-1 > first_last6

Given an array of ints, return True if 6 appears as either the first or last element in the array. The array will be length 1 or more.

first_last6([1, 2, 6]) → True
first_last6([6, 1, 2, 3]) → True
first_last6([13, 6, 1, 2, 3]) → False

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def first_last6(nums):
    return nums[0] == 6 or nums[-1] == 6
```

Go

Expected	Run		
first_last6([1, 2, 6]) → True	True	OK	
first_last6([6, 1, 2, 3]) → True	True	OK	
first_last6([13, 6, 1, 2, 3]) → False	False	OK	
first_last6([13, 6, 1, 2, 6]) → True	True	OK	
first_last6([3, 2, 1]) → False	False	OK	
first_last6([3, 6, 1]) → False	False	OK	
first_last6([3, 6]) → True	True	OK	
first_last6([6]) → True	True	OK	
first_last6([3]) → False	False	OK	
first_last6([5, 6]) → True	True	OK	
first_last6([5, 5]) → False	False	OK	
first_last6([1, 2, 3, 4, 6]) → True	True	OK	
first_last6([1, 2, 3, 4]) → False	False	OK	
other tests		OK	

All Correct

List-1 > same_first_last

Given an array of ints, return True if the array is length 1 or more, and the first element and the last element are equal.

same_first_last([1, 2, 3]) → False
same_first_last([1, 2, 3, 1]) → True
same_first_last([1, 2, 1]) → True

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def same_first_last(nums):
    return len(nums) >= 1 and nums[0] == nums[-1]
```

Expected	Run		
same_first_last([1, 2, 3]) → False	False	OK	
same_first_last([1, 2, 3, 1]) → True	True	OK	
same_first_last([1, 2, 1]) → True	True	OK	
same_first_last([7]) → True	True	OK	
same_first_last([]) → False	False	OK	
same_first_last([1, 2, 3, 4, 5, 1]) → True	True	OK	
same_first_last([1, 2, 3, 4, 5, 13]) → False	False	OK	
same_first_last([13, 2, 3, 4, 5, 13]) → True	True	OK	
same_first_last([7, 7]) → True	True	OK	
other tests		OK	

All Correct

Good job -- problem solved. You can see our solution as an alternative.

See Our Solution

List-1 > make_pi

Return an int array length 3 containing the first 3 digits of pi, {3, 1, 4}.

make_pi() → [3, 1, 4]

Go ...Save, Compile, Run (ctrl-enter)

```
def make_pi():
    return [3, 1, 4]
```

Expected	Run		
make_pi() → [3, 1, 4]	[3, 1, 4]	OK	

All Correct

next | chance

Python > List-1

federror.tim@gmail.com done page

Your progress graph for this problem

List-1 > common_end

Given 2 arrays of ints, a and b, return True if they have the same first element or they have the same last element. Both arrays will be length 1 or more.

common_end([1, 2, 3], [7, 3]) → True
common_end([1, 2, 3], [7, 3, 2]) → False
common_end([1, 2, 3], [1, 3]) → True

Go ...Save, Compile, Run (ctrl-enter)

```
def common_end(a, b):
    return a[0] == b[0] or a[-1] == b[-1]
```

Expected	Run		
common_end([1, 2, 3], [7, 3]) → True	True	OK	
common_end([1, 2, 3], [7, 3, 2]) → False	False	OK	
common_end([1, 2, 3], [1, 3]) → True	True	OK	
common_end([1, 2, 3], [1]) → True	True	OK	
common_end([1, 2, 3], [2]) → False	False	OK	
other tests		OK	

All Correct

List-1 > sum3

Given an array of ints length 3, return the sum of all the elements.

sum3([1, 2, 3]) → 6
sum3([5, 11, 2]) → 18
sum3([7, 0, 0]) → 7

Go ...Save, Compile, Run (ctrl-enter)

```
def sum3(nums):
    return sum(nums)
```

Expected	Run		
sum3([1, 2, 3]) → 6	6	OK	
sum3([5, 11, 2]) → 18	18	OK	
sum3([7, 0, 0]) → 7	7	OK	
sum3([1, 2, 1]) → 4	4	OK	
sum3([1, 1, 1]) → 3	3	OK	
sum3([2, 7, 2]) → 11	11	OK	

All Correct

List-1 > rotate_left3

Given an array of ints length 3, return an array with the elements "rotated left" so {1, 2, 3} yields {2, 3, 1}.

rotate_left3([1, 2, 3]) → [2, 3, 1]
rotate_left3([5, 11, 9]) → [11, 9, 5]
rotate_left3([7, 0, 0]) → [0, 0, 7]

Go ...Save, Compile, Run (ctrl-enter)

```
def rotate_left3(nums):
    return nums[1:] + nums[:1]
```

Expected	Run		
rotate_left3([1, 2, 3]) → [2, 3, 1]	[2, 3, 1]	OK	
rotate_left3([5, 11, 9]) → [11, 9, 5]	[11, 9, 5]	OK	
rotate_left3([7, 0, 0]) → [0, 0, 7]	[0, 0, 7]	OK	
rotate_left3([1, 2, 1]) → [2, 1, 1]	[2, 1, 1]	OK	
rotate_left3([0, 0, 1]) → [0, 1, 0]	[0, 1, 0]	OK	
other tests		OK	

All Correct

List-1 > reverse3

prev | next | chance

Given an array of ints length 3, return a new array with the elements in reverse order, so {1, 2, 3} becomes {3, 2, 1}.

reverse3([1, 2, 3]) → [3, 2, 1]
reverse3([5, 11, 9]) → [9, 11, 5]
reverse3([7, 0, 0]) → [0, 0, 7]

Go ...Save, Compile, Run (ctrl-enter)

```
def reverse3(nums):
    return nums[::-1]
```

Expected	Run		
reverse3([1, 2, 3]) → [3, 2, 1]	[3, 2, 1]	OK	
reverse3([5, 11, 9]) → [9, 11, 5]	[9, 11, 5]	OK	
reverse3([7, 0, 0]) → [0, 0, 7]	[0, 0, 7]	OK	
reverse3([2, 1, 2]) → [2, 1, 2]	[2, 1, 2]	OK	
reverse3([1, 2, 1]) → [1, 2, 1]	[1, 2, 1]	OK	
reverse3([2, 11, 3]) → [3, 11, 2]	[3, 11, 2]	OK	
reverse3([0, 6, 5]) → [5, 6, 0]	[5, 6, 0]	OK	
reverse3([7, 2, 3]) → [3, 2, 7]	[3, 2, 7]	OK	
other tests		OK	

All Correct

List-1 > max_end3

Given an array of ints length 3, figure out which is larger, the first or last element in the array, and set all the other elements to be that value. Return the changed array.

max_end3([1, 2, 3]) → [3, 3, 3]
max_end3([11, 5, 9]) → [11, 11, 11]
max_end3([2, 11, 3]) → [3, 3, 3]

Go ...Save, Compile, Run (ctrl-enter)

```
def max_end3(nums):
    m = max(nums[0], nums[-1])
    return [m, m, m]
```

Expected	Run		
max_end3([1, 2, 3]) → [3, 3, 3]	[3, 3, 3]	OK	
max_end3([11, 5, 9]) → [11, 11, 11]	[11, 11, 11]	OK	
max_end3([2, 11, 3]) → [3, 3, 3]	[3, 3, 3]	OK	
max_end3([11, 3, 3]) → [11, 11, 11]	[11, 11, 11]	OK	
max_end3([3, 11, 11]) → [11, 11, 11]	[11, 11, 11]	OK	
max_end3([2, 2, 2]) → [2, 2, 2]	[2, 2, 2]	OK	
max_end3([2, 11, 2]) → [2, 2, 2]	[2, 2, 2]	OK	
max_end3([0, 0, 1]) → [1, 1, 1]	[1, 1, 1]	OK	
other tests		OK	

All Correct

List-1 > sum2

Given an array of ints, return the sum of the first 2 elements in the array. If the array length is less than 2, just sum up the elements that exist, returning 0 if the array is length 0.

sum2([1, 2, 3]) → 3
sum2([1, 1]) → 2
sum2([1, 1, 1, 1]) → 2

Go ...Save, Compile, Run (ctrl-enter)

```
def sum2(nums):
    return sum(nums[:2])
```

Expected	Run		
sum2([1, 2, 3]) → 3	3	OK	
sum2([1, 1]) → 2	2	OK	
sum2([1, 1, 1, 1]) → 2	2	OK	
sum2([1, 2]) → 3	3	OK	
sum2([1]) → 1	1	OK	
sum2([]) → 0	0	OK	
sum2([4, 5, 6]) → 9	9	OK	
sum2([4]) → 4	4	OK	
other tests		OK	

List-1 > middle_way

Given 2 int arrays, a and b, each length 3, return a new array length 2 containing their middle elements.

middle_way([1, 2, 3], [4, 5, 6]) → [2, 5]
middle_way([7, 7, 7], [3, 8, 0]) → [7, 8]
middle_way([5, 2, 9], [1, 4, 5]) → [2, 4]

Go ...Save, Compile, Run (ctrl-enter)

```
def middle_way(a, b):
    return [a[1], b[1]]
```

Expected	Run		
middle_way([1, 2, 3], [4, 5, 6]) → [2, 5]	[2, 5]	OK	
middle_way([7, 7, 7], [3, 8, 0]) → [7, 8]	[7, 8]	OK	
middle_way([5, 2, 9], [1, 4, 5]) → [2, 4]	[2, 4]	OK	
middle_way([1, 9, 7], [4, 8, 8]) → [9, 8]	[9, 8]	OK	
middle_way([1, 2, 3], [3, 1, 4]) → [2, 1]	[2, 1]	OK	
middle_way([1, 2, 3], [4, 1, 1]) → [2, 1]	[2, 1]	OK	
other tests		OK	

All Correct

List-1 > make_ends

Given an array of ints, return a new array length 2 containing the first and last elements from the original array. The original array will be length 1 or more.

make_ends([1, 2, 3]) → [1, 3]
make_ends([1, 2, 3, 4]) → [1, 4]
make_ends([7, 4, 6, 2]) → [7, 2]

Go ...Save, Compile, Run (ctrl-enter)

```
def make_ends(nums):
    return [nums[0], nums[-1]]
```

Expected	Run		
make_ends([1, 2, 3]) → [1, 3]	[1, 3]	OK	
make_ends([1, 2, 3, 4]) → [1, 4]	[1, 4]	OK	
make_ends([7, 4, 6, 2]) → [7, 2]	[7, 2]	OK	
make_ends([1, 2, 2, 2, 2, 2, 2, 3]) → [1, 3]	[1, 3]	OK	
make_ends([7, 4]) → [7, 4]	[7, 4]	OK	
make_ends([7]) → [7, 7]	[7, 7]	OK	
make_ends([5, 2, 9]) → [5, 9]	[5, 9]	OK	
make_ends([2, 3, 4, 1]) → [2, 1]	[2, 1]	OK	
other tests		OK	

All Correct

List-1 > has23

Given an int array length 2, return True if it contains a 2 or a 3.

has23([2, 5]) → True
has23([4, 3]) → True
has23([4, 5]) → False

Go ...Save, Compile, Run (ctrl-enter)

```
def has23(nums):
    return 2 in nums or 3 in nums
```

Expected	Run		
has23([2, 5]) → True	True	OK	
has23([4, 3]) → True	True	OK	
has23([4, 5]) → False	False	OK	
has23([2, 2]) → True	True	OK	
has23([3, 2]) → True	True	OK	
has23([3, 3]) → True	True	OK	
has23([7, 7]) → False	False	OK	
has23([3, 9]) → True	True	OK	
has23([9, 5]) → False	False	OK	
other tests		OK	

All Correct